

Изключения - механизъм за обработка на грешки при необичайни ситуации, които: не са част от нормалния контролен ~~блок~~^{поток}, не могат/не е удобно да се обработват локално.

⊕ мощни са
⊖ скрити и скъпи

Прекъсва се текущото изпълнение, контрола се прехвърля нагоре по call stack-a

Примери: Неуспешно заделяне на памет, невалиден вход, файл не може да се отвори, логическа грешка

throw - хвърля обект, който се копира/премества в специална exception област. Типът на обекта определя кой catch блок ще го улови

```
try {
    // дефинира защитена зона
    // блок, в който може да настъпи изключението
}
catch (const std::invalid_argument & e) {
    // редът е критичен от най-специфичния към най-общия
}
catch (const std::exception & e) {
    // улавя изключения по тип
}
catch (...) {
    // изпълнява се първия съвпадащ тип
    // - улавя всяко изключение
}
```

Когато се хвърли изключение:

1. Текущата функция прекъсва
2. Започва размотаване на стека
3. Всички локални обекти се унищожават
4. Търси се първия catch, който може да улови exception.

При улавянето на exception в catch трябва да бъде по референция/const референция, а не по стойност (ще имаме object slicing)

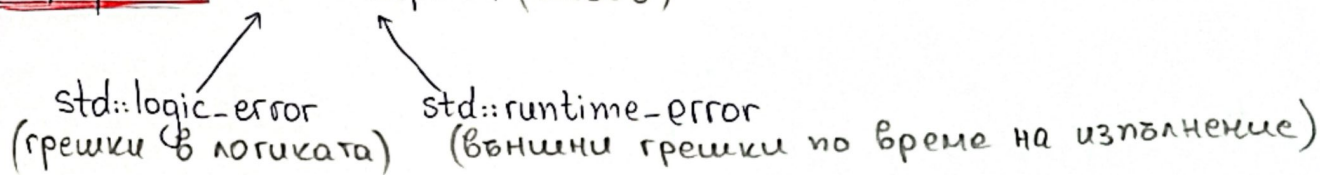
Използване изключения:

- грешки, които не са част от нормалната логика
- конструктори (нямат return)
- нарушения на класа

std::current_exception & getthrow_exception

- Прехвърля, запази или логи, после повторно хвърли

Иерархия: std::exception (базов)



Изключения в конструктор - когато обектът не може да бъде валидно

Ако конструктор хвърли:

- обектът не е създаден и деструктора му не се вика
- деструкторите на вече конструирани обекти се извикват автоматично
- Ако обект не може да съществува в невалидно състояние - хвърля в конструктор

Изключения в деструктора - не трябва, те са поехсерт (true) по default

- Ако деструктора хвърли по време на размотаването на стека (вече има активно изключение) → std::terminate()

Нива на exception safety

1. No-throw - операцията не хвърля; поехсерт; при деструктори, swap, move
2. Strong-guarantee - операцията завършва успешно или състоянието на програмата остава напълно непроменено; работа върху временни обекти
3. Basic guarantee - след изключение данните на обекта са запазени, състоянието може да е променено (без изтичане на ресурси)
4. No guarantee - никаква гаранция

enum class ErrorCode - функцията връща стойност, която указва ^{успех} ~~стойност~~ или тип грешка

⊕ предсказуем контролен поток

⊖ примитивни са

⊕ при специално поведение на наши системи

⊕ съвпада бизнес логиката с грешките

⊕ може да се пропусне грешка или да се изтрие по грешка

std::expected - тип, съдържащ валидна стойност T или грешка E

std::expected<int, ErrorCode> - int при успех, ErrorCode при грешка

⊕ явни грешки

⊕ без runtime overhead

⊕ модерен API